

Chess Outcome Prediction

Big Data Pipeline from Move-Level Board States

Ivan Chabanov, Alexander Mikhailov, Danil Eramasov, Bulat Fakhrutdinov

Big Data Course -- IU, 2026

1. Project Goal

We build a scalable Big Data system for chess analytics.

Main task:

```
board state + game context at a selected ply  
-> white_win | black_win | draw
```

The project covers the full path from raw Chess.com archives to distributed storage, EDA, dashboard, and a reserved ML section.

2. Dataset

Source: Chess.com public archive API.

Unit of observation: **one ply**: one half-move from one game.

Property	Value
Rows	674,594 plies
Size	589.22 MB
Features	100+
Target	final_result_class
Classes	white_win, black_win, draw

The dataset satisfies the course scale requirements.

3. Feature Space

The dataset combines game context and chess-specific board features.

Group	Examples
Game metadata	players, ratings, time control, ECO opening
Move context	SAN/UCI move, side to move, ply index
Board state	FEN, material, legal move count, phase
Derived features	pawn structure, king safety, castling, time pressure
Target	final game outcome

Leakage columns such as raw result and termination are kept for audit only.

4. Pipeline Architecture

```
graph TD; A[Chess.com PGN archives] --> B[CSV generation and validation]; B --> C[PostgreSQL]; C --> D[Sqoop]; D --> E[HDFS / Parquet / Snappy]; E --> F[Hive warehouse]; F --> G[EDA + Superset + Spark ML template];
```

Chess.com PGN archives
|
CSV generation and validation
|
PostgreSQL
|
Sqoop
|
HDFS / Parquet / Snappy
|
Hive warehouse
|
EDA + Superset + Spark ML template

This is the core Big Data part of the project.

5. Storage Design

The main analytical table is `chess_moves`.

Layer	Decision
PostgreSQL	schema and initial ingestion
HDFS	distributed storage
Hive	external warehouse table
Format	Parquet
Compression	Snappy
Partitioning	<code>archive_month</code>
Bucketing	<code>game_uuid</code> , 8 buckets

The denormalized ply-level table is easier for Spark and Hive scans.

6. EDA Deliverables

Stage II produces eight HiveQL insights.

Query	Question
q1	outcome by time control
q2	top ECO openings by white win rate
q3	rating gap vs outcome
q4	game length by time control
q5	checkmate phase distribution
q6	outcome trends over time
q7	outcome by termination type
q8	material advantage vs outcome

Game-level insights use `ply_index = 1` to avoid over-weighting long games.

7. Dashboard

Apache Superset is the presentation layer.

Each insight has:

- Hive result table
- exported CSV artifact
- chart in Superset
- dashboard-level interpretation

The dashboard focuses on ratings, openings, time controls, material advantage, game phase, and temporal trends.

8. ML Template

The current report does not contain final ML results yet. This slide is a placeholder for the Spark MLlib part once model training and evaluation are complete.

Planned setup:

Item	Plan
Task	multiclass classification
Target	final_result_class
Split	train/test by game_uuid
Models	Random Forest, SVM / One-vs-Rest, Naive Bayes
Metrics	accuracy, weighted F1, confusion matrix
Baseline	majority class on train split

9. Final Takeaway

What is ready now:

1. Chess.com data is transformed into a 674k-row ply-level dataset.
2. The project implements PostgreSQL -> Sqoop -> HDFS -> Hive.
3. Hive storage uses Parquet, Snappy, partitioning, and bucketing.
4. Eight EDA queries support the Superset dashboard.
5. The ML section has a clean template and should be filled after Spark MLlib results are available.

This version is short enough for a defense talk and leaves room for the future ML slide update.